This document provides a comprehensive guide for partners (merchants) to integrate Mercuryo's Native Apple Pay solution. This allows you to offer a seamless Apple Pay checkout experience directly on your website or in your application, while Mercuryo handles the payment processing.

- Integration Flow Overview
    - Sequence Diagram
- Onboarding: Certificate Setup
    - 1. Merchant Identity Certificate
    - 2. Payment Processing Certificate
- Step 1: Getting a Quote
    - API Endpoint: GET /widget/buy/rate
    - Success Response
- Step 2: Partner-Side Implementation
    - Frontend: Initiating the Payment Request
    - Backend: Validating the Merchant Session
- Step 3: Calling the Mercuryo API
    - API Endpoint: POST /native-mobile-pay
        - Request Body
        - Success Response
- Handling Errors and Fallback to Widget
    - Triggering the Fallback
    - Partner Frontend Implementation
- Step 4: Receiving the Final Status (Callback)
    - Callback Verification
    - Callback Payload
- Important Considerations

Integration Flow Overview

The integration is based on a "headless" model where your application manages the user interface and initial interaction with Apple, and Mercuryo's backend manages the payment execution.

The process consists of three main stages:

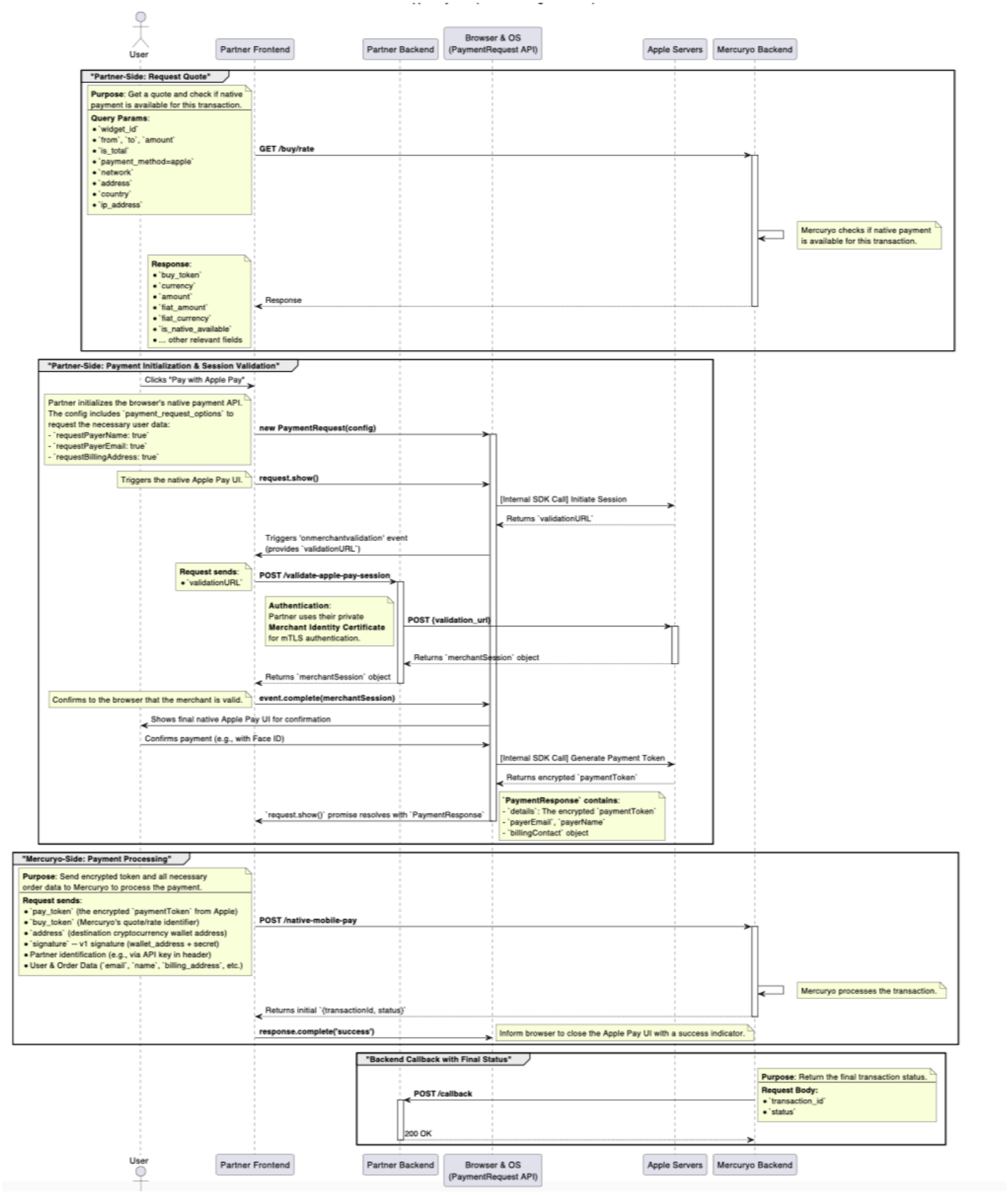
1. **Client-Side (Partner's Frontend)**: The user initiates the payment, and your frontend application communicates with the Apple Pay SDK.
2. **Server-Side (Partner's Backend)**: Your backend validates the merchant session with Apple using your

own `Merchant Identity Certificate`.

3. **Processing (Mercuryo's Backend)**: Your application sends the encrypted payment data to Mercuryo, where we decrypt it using your `Payment Processing Certificate` and process the transaction.

Sequence Diagram

This diagram provides a detailed, step-by-step visualization of the entire flow.



Onboarding: Certificate Setup

Before you can begin the integration, you must configure two types of Apple Pay certificates. This integration relies on a two-certificate model to ensure maximum security and clear separation of responsibilities.

1. Merchant Identity Certificate

- **Purpose**: To authenticate your server with Apple's servers when validating a merchant session. This certificate is used to establish a secure TLS connection for the session validation request.
- **Responsibility**: **Yours**.
- **Action**: You must generate this certificate in your Apple Developer account and securely store it along with its private key on your backend infrastructure. You will use it to handle the `onmerchantvalidation` step. Mercuryo **never** sees this certificate or its key.
- **Official Documentation**: For details, see Apple's documentation on Creating a Merchant Identity Certificate.

2. Payment Processing Certificate

- **Purpose**: To allow Mercuryo to decrypt the final payment token. Apple uses the public key of this certificate to encrypt the payment data.
- **Responsibility**: **Yours to generate, ours to use**.
- **Action**: You must also generate this certificate in your Apple Developer account. During the onboarding process, you must **securely provide Mercuryo with both the certificate file (`.cer`) and its corresponding private key (`.key`)**. We will store these credentials securely and use them to process payments on your behalf.
- **Official Documentation**: This certificate is associated with your Merchant ID and is used for encrypting payment data. See Apple's guide on configuring Apple Pay.

For a detailed walkthrough of how to generate and manage these certificates, please refer to the **Certificate Management Guide**.

Step 1: Getting a Quote

Before initiating the Apple Pay payment sheet, your application must first get a transaction quote from Mercuryo. This step provides your user with the exact exchange rate and provides your application with a `buy_token` required to execute the payment.

You must call the standard `/widget/buy/rate` endpoint. To specify that this is a Native Apple Pay transaction, you must include the `payment_method=apple` parameter.

API Endpoint: `GET /widget/buy/rate`

Parameter	Required For Native Flow Determination	Description
`widget_id`	**Yes**	Your unique identifier provided by Mercuryo.
`from`	**Yes**	Source currency code (e.g., `USD`).
`to`	**Yes**	Target currency code (e.g., `BTC`).
`amount`	**Yes**	The amount in the `from` currency.
`payment_method`	**Yes**	Must be `apple` for this flow.
`is_total`	**Yes**	A boolean that determines how to interpret the `amount`. While optional in the generic API, it is **required** for this flow to ensure correct calculations. Defaults to `true`.
`network`	**Yes**	While optional in the generic API, the blockchain network is **required** for this flow to determine availability and correct fees.
`address`	**Yes**	While optional in the generic API, the destination wallet address is **required** for this flow for risk assessment.
`country` / `ip_address`	**Yes**	At least one of these fields is **required** to perform geo-restriction checks. `ip_address` is the preferred method.

Success Response

The response will contain the current rate, fee breakdown, and the `buy_token`. Crucially, it will also include a new boolean flag, `is_native_available`.

```
{    "status": 200,    "data": {        "buy_token": "abc123def456...",        "currency":
"BTC",        "amount": "0.00234398",        "fiat_amount": "300.00",
"fiat_currency": "USD",      ...        "is_native_available": true    } }
```

Your frontend should check for `is_native_available: true`. If this flag is present and `true`, you can proceed with showing the native Apple Pay button. If it is `false` or absent, you should fall back to a different flow (e.g., redirecting the user to the full Mercuryo widget).

You must store the `buy_token` as it is a **required** field for the final `POST /native-mobile-pay` call.

Step 2: Partner-Side Implementation

Frontend: Initiating the Payment Request

When the user clicks the "Pay with Apple Pay" button, your frontend must create an `ApplePayPaymentRequest`. It is crucial that this request asks for all the necessary user information that Mercuryo requires for processing. For the full specification of this object, see Apple's official documentation for ApplePayPaymentRequest.

Example `PaymentRequest` Configuration:

```
const paymentRequest = {    // ... your countryCode, currencyCode, total, etc.
"payment_request_options": {        "requestShipping": false,        "requestPayerName":
true,        "requestPayerEmail": true,        "requestPayerPhone": false,
"requestBillingAddress": true    },    // ... other Apple Pay configuration };
```

Your frontend will then handle the `onmerchantvalidation` event by calling your backend to get a valid `merchantSession`. After the user authorizes the payment, your frontend will receive the encrypted `paymentToken`.

Backend: Validating the Merchant Session

Your backend must expose an endpoint that your frontend can call during the `onmerchantvalidation` event. This endpoint is responsible for:

1. Receiving the `validationURL` from your frontend.
2. Making a `POST` request to that URL using your `Merchant Identity Certificate` to authenticate with Apple's servers.
3. Returning the resulting `merchantSession` object to your frontend.

Step 3: Calling the Mercuryo API

Once your frontend has the encrypted `paymentToken`, it sends this token along with other transaction details to Mercuryo's dedicated native processing endpoint.

API Endpoint: `POST /native-mobile-pay`

- **Authentication**: The request must be authenticated using your partner API Key, sent as an HTTP header.
- **Content-Type**: `application/json`

Request Body

Field	Type	Description
`pay_token`	`string`	**Required.** The encrypted `paymentToken` received from the Apple Pay SDK.
`buy_token`	`string`	**Required.** The unique quote identifier received from Mercuryo's `GET /widget/buy/rate` endpoint.
`email`	`string`	**Required.** The user's email address, obtained from the Apple Pay payment sheet.
`first_name`	`string`	**Required.** The user's first name.
`last_name`	`string`	**Required.** The user's last name.
`billing_address`	`object`	**Required.** The user's billing address object, as provided by Apple Pay. The object must match the structure of Apple's `ApplePayPaymentContact` object. See example below.
`address`	`string`	**Required.** The destination cryptocurrency wallet address for the user.
`merchant_transaction_id`	`string`	*Optional.* Your internal identifier for the transaction.
`signature`	`string`	**Required.** The V1 signature.

Example `billing_address` object:

```
"billing_address": {    "country_code": "US",    "street_line_1": "Asden Ct",
"street_line_2": "123",    "state_code": "OH",    "city": "Рейнолдсбург",    "zip_code":
"43068" }
```

Success Response

Mercuryo's endpoint will synchronously return the initial status of the transaction in a nested `data` object.

```
{    "status": 200,    "data": {        "id": "0e868c530209b7813",        "status":
"order_scheduled",        "payment_status": "authorized",        "currency": "BTC",
"amount": "0.00234398",        "fiat_currency": "USD",        "fiat_amount": "300.00",
"address": "2Mth4myXTo9RMjZKNdcs6q58ox25jC1Qpwi",        "transaction": {            "id":
"0e868c5309b382234"        }    } }
```

The `status` field inside the `data` object (e.g., `"order_scheduled"`) indicates that the transaction has been accepted and is being processed. The final status will be sent via the server-to-server callback.

Handling Errors and Fallback to Widget

While the API can return standard HTTP error codes for general issues (e.g., `400 Bad Request` for invalid parameters), the most critical response to handle is the one that triggers a fallback flow.

A key feature of this integration is the ability to seamlessly transition the user from a native Apple Pay flow to the full Mercuryo widget if additional user action is required. This ensures that transactions are not lost in cases where simple native payment is insufficient (e.g., the user needs to complete KYC).

Triggering the Fallback

The fallback mechanism is triggered when the `/native-mobile-pay` endpoint responds with a specific error code indicating that the transaction cannot be completed natively. This response indicates that while the request is valid, it cannot be processed without further user interaction. A detailed list of error codes will be provided later.

Example Fallback Response:

```
{    "status": 4xx,    "code": <error_code>,    "message": "User action required to complete the transaction.",    "data": {        "reason": "KYC_REQUIRED",        "redirect_url": "https://widget.mercuryo.io/?buy_token=..."    } }
```

- `reason`: A code indicating why the fallback is necessary (e.g., `KYC_REQUIRED`, `TURNOVER_LIMIT_EXCEEDED`).
- `redirect_url`: A unique, single-use URL to the Mercuryo widget.

Partner Frontend Implementation

Your frontend application should be designed to handle this fallback response:

1. Check the response for the specific error code that signals a fallback is needed.
2. If it is present, extract the `redirect_url` from the response body.
3. Redirect the user's browser to this URL.

The user will land on the Mercuryo widget with all their transaction data pre-filled, where they can complete the required steps (e.g., upload documents for KYC) and finalize their purchase.

Step 4: Receiving the Final Status (Callback)

Because the final processing of the transaction is asynchronous, Mercuryo will notify your backend of the final transaction status using our standard server-to-server webhook. You must provide a secure callback URL to your Mercuryo integration manager during onboarding.

Callback Verification

To ensure the security and authenticity of the callback, each request will contain an `x-signature` HTTP header. You must verify this signature by calculating an HMAC-SHA256 hash of the raw JSON payload using your widget's `Sign key` (available in your Mercuryo dashboard) and comparing it to the value of the header.

Callback Payload

The callback payload is a standard `CallbackNotification` object, as described in our main API reference. For a Native Apple Pay transaction, the `data` object within the payload will have the following key identifiers:

- `"type": "buy"`
- `"payment_method": "mobile_pay"`
- `"method_code": "apple"`

The `status` field within the `data` object will reflect the final state of the transaction, such as `paid`, `order_failed`, or `cancelled`.

Please refer to the `/callback` section in our main `swagger-widget-integration.yaml` documentation for the complete schema of the callback body and all possible transaction statuses.

Important Considerations

- **Country Restrictions**: This flow is **not** available for users whose billing country is the **United Kingdom (GB)** or the **United States (US)**. Mercuryo will enforce this on our backend, but we recommend you also apply this logic on your frontend to improve the user experience.
- **Transaction Limits for Non-KYC Users**: For users who have not completed the KYC (Know Your Customer) procedure, there is a cumulative transaction limit. The total turnover for a user, calculated across all their transactions and converted to EUR, cannot exceed €700. If a new transaction causes the user to exceed this limit, it will be rejected. This is one of the primary scenarios that may trigger an error requiring the user to be redirected to the full Mercuryo widget to complete KYC.